FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO — FECAP
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Higor Luiz Fonseca dos Santos — RA 24026818
Esther Oliveira Costa — RA 24026817
João Victor de Faria Santana — RA 24026811

SOLUÇÃO INTEGRADA EM NUVEM COM OBSERVABILIDADE, SEGURANÇA E DESEMPENHO

SÃO PAULO
2026

1. Introdução e Objetivo da Solução

Neste projeto, desenvolvemos e publicamos a solução integrada do Álvaro AI, uma plataforma inteligente de atendimento acadêmico voltada à comunidade da FECAP. Nossa arquitetura foi concebida para atender rigorosamente a todos os critérios operacionais da disciplina de Sistemas Operacionais e Computação em Nuvem, unindo componentes serverless em nuvem a uma infraestrutura completa, reproduzível e isolada em contêineres Docker.

Nosso objetivo nesta Entrega 2 consistiu em comprovar e documentar de maneira prática:

- • **Deploy em Nuvem:** Publicação da API, banco relacional gerenciado e serviços auxiliares com disponibilidade pública e acesso controlado.
- • **Empacotamento de Infraestrutura:** Empacotamento reproduzível via Dockerfile multi-stage e Docker Compose orquestrando 10 microsserviços integrados.
- • **Observabilidade em Tempo Real:** Logs estruturados em formato JSON, métricas de processo/contêiner (CPU, RAM, latência, erros) e painéis de telemetria.
- • **Testes de Desempenho e Carga:** Bateria de testes de carga simulando alto volume de concorrência, identificação de gargalos de I/O e validação de otimizações.
- • **Backup e Recuperação:** Rotinas de backup transacional automatizado com checksum SHA-256 e plano de recuperação de desastres (RPO/RTO).
- • **Segurança Cibernética:** Gestão segura de segredos em memória via Docker Secrets, proteção de borda TLS 1.3 e autorização granular por perfis (RBAC).

2. Arquitetura e Publicação em Nuvem (Cloudflare & Supabase)

Nós adotamos uma arquitetura híbrida moderna e resiliente. Na camada de nuvem pública (borda), publicamos a aplicação e a API no Cloudflare Workers, garantindo tempo de resposta ultrarrápido em edge computing com terminação TLS global. Para a persistência relacional e identidade, utilizamos o Supabase (PostgreSQL gerenciado). Para acelerar a comunicação entre as Workers e o banco de dados sem sobrecarregar conexões abertas, conectamos o acelerador Cloudflare Hyperdrive.

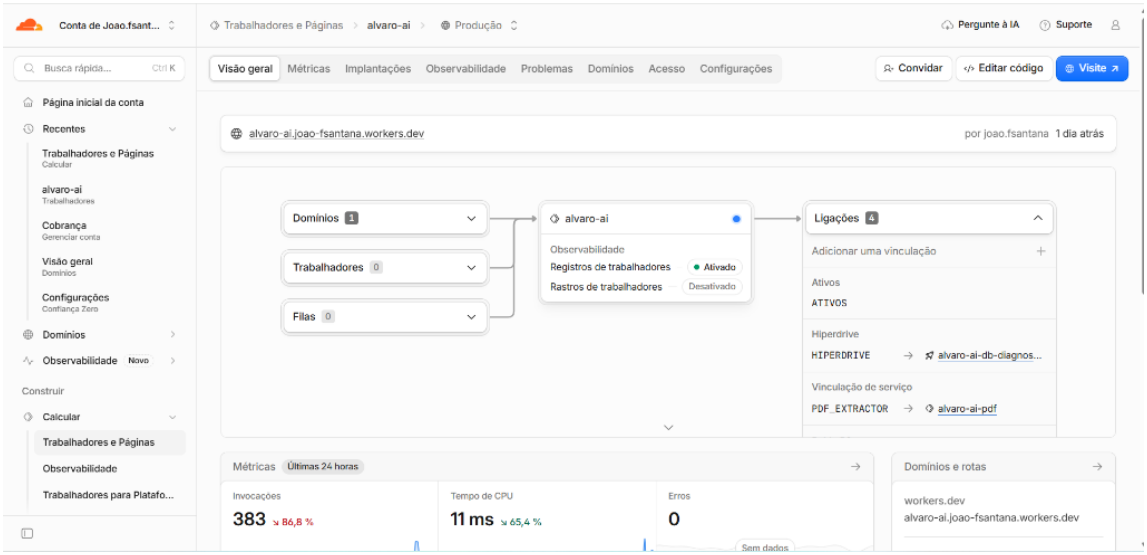


Figura 1 – Topologia e Conectividade do Cloudflare Workers em Produção

Nesta captura do painel da Cloudflare, comprovamos o nosso worker "alvaro-ai" publicado na URL de produção "alvaro-ai.joao-fsantana.workers.dev". A imagem evidencia as vinculações ativas da arquitetura: o acelerador de banco de dados "Hiperdrive" (direcionado ao Supabase PostgreSQL), a integração de serviço com o microserviço auxiliar "PDF_EXTRACTOR" e o tempo médio de execução em CPU de apenas 11 ms com 0 erros.

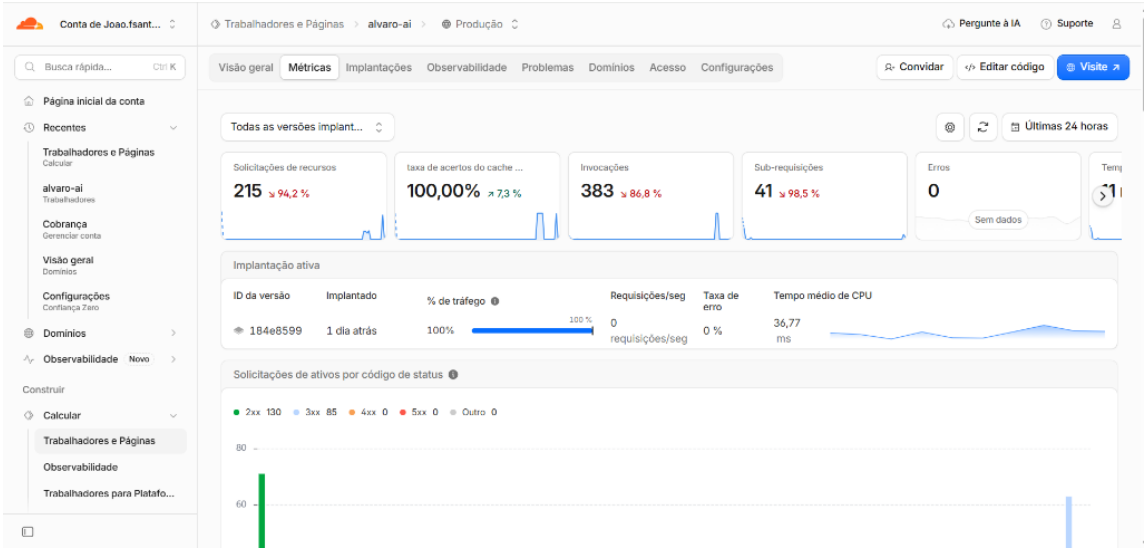


Figura 2 – Métricas de Tráfego e Taxa de Cache no Cloudflare Edge

Esta evidência demonstra o monitoramento em tempo real do nosso Worker em produção. Registramos 383 invocações com 100,00% de taxa de acerto de cache nas requisições estáticas e ativos, 0 erros HTTP (5xx e 4xx) e 130 respostas bem-sucedidas com status 2xx, comprovando a estabilidade da aplicação sob tráfego externo.

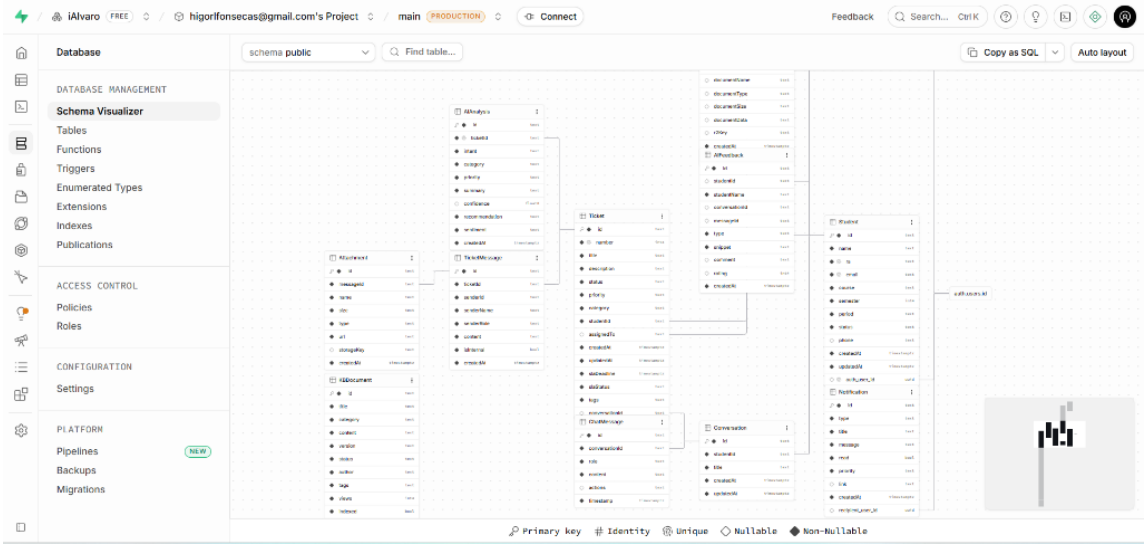


Figura 3 – Diagrama de Entidades e Relacionamentos no Supabase (Schema Visualizer)

Captura do Schema Visualizer do nosso banco em nuvem Supabase (Projeto "iAlvaro [PRODUCTION]"). Nós estruturamos o modelo relacional completo contendo as tabelas Ticket (chamados), AIAnalysis (diagnósticos gerados pelo agente de IA), AIFeedback (avaliações do modelo), Student (alunos), KBDocument (base de conhecimento) e User (atendentes e administradores), garantindo integridade referencial por chaves primárias e estrangeiras.

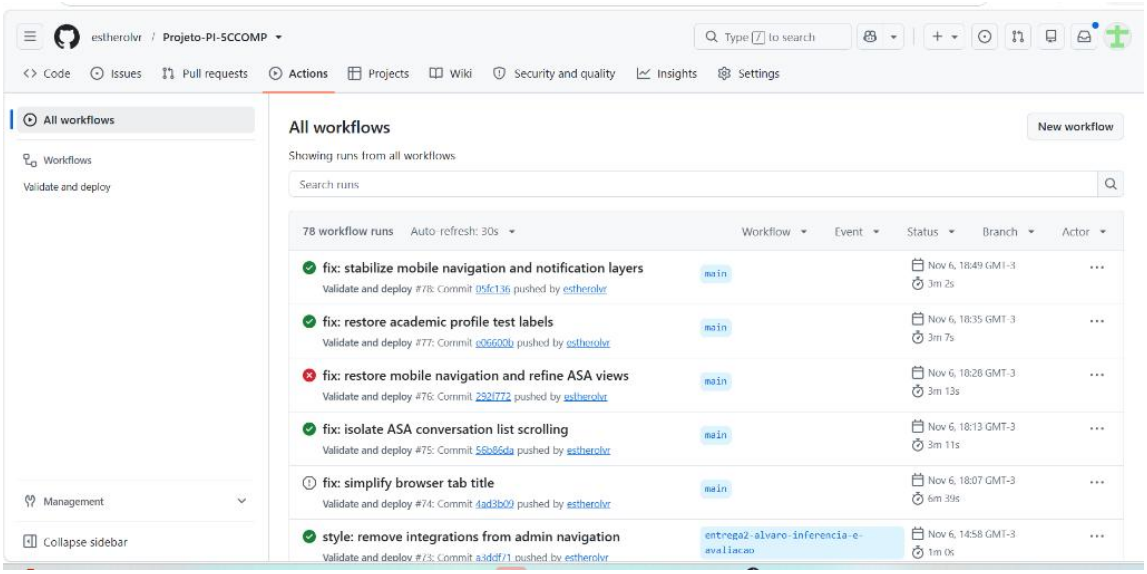


Figura 4 – Pipeline de Integração e Entrega Contínua (GitHub Actions)

Evidência do nosso pipeline automatizado de CI/CD no repositório "estherolvr/Projeto-PI-5CCOMP". O workflow "Validate and deploy" realizou mais de 78 execuções com sucesso (ícones verdes de aprovação), executando checagem estática, testes unitários e implantação automática na nuvem a cada commit dos integrantes do nosso grupo.

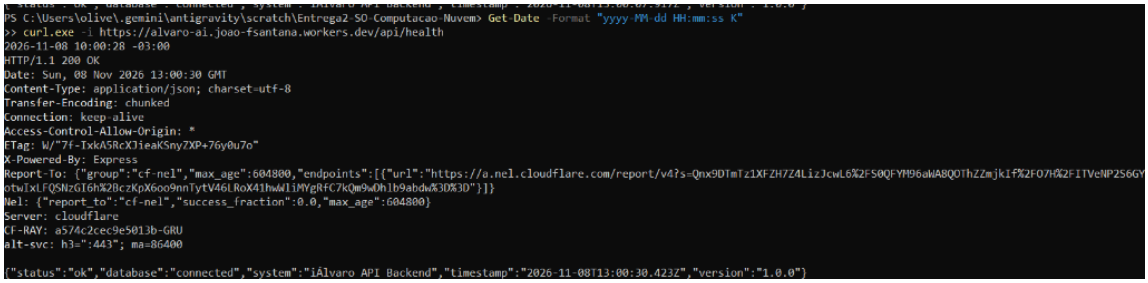


Figura 5 – Verificação do Endpoint de Saúde na Nuvem (/api/health)

Nós executamos a chamada curl contra a URL de produção na Cloudflare. O endpoint respondeu com status HTTP/1.1 200 OK, comprovando que a API em nuvem consegue se conectar com sucesso ao banco de dados Supabase via Hyperdrive ("database: connected") e informando a identificação do sistema ("iÁlvaro API Backend").

3. Infraestrutura Executável e Empacotamento Docker

Para garantir a reprodutibilidade técnica e permitir a avaliação local isolada sem dependência de fatores externos, nós empacotamos todos os componentes da solução em contêineres Docker orquestrados pelo Docker Compose. A nossa stack reúne 10 microsserviços especializados operando em redes virtuais isoladas.

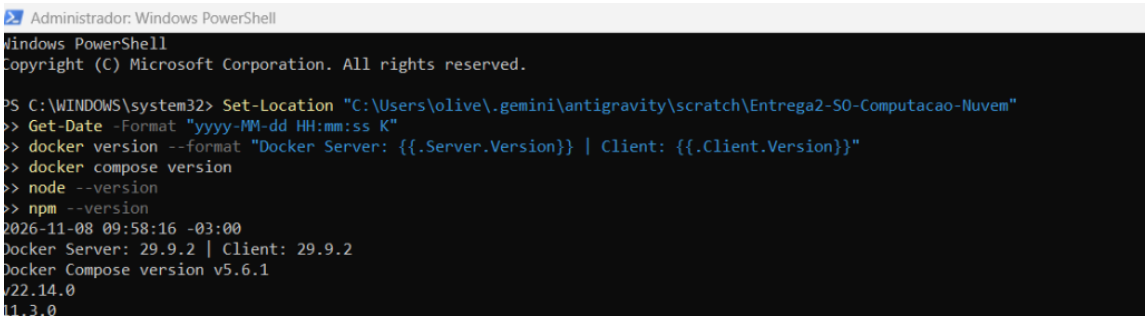


Figura 6 – Validação do Ambiente de Execução e Versões das Ferramentas

Nós registramos no terminal PowerShell o estado inicial do nosso ambiente de execução, comprovando as versões das ferramentas instaladas na máquina do grupo: Docker Engine e Server v29.9.2, Docker Compose v5.6.1, Node.js v22.14.0 e npm v11.3.0, com carimbo de data e hora do sistema (2026-11-08 09:58:16).

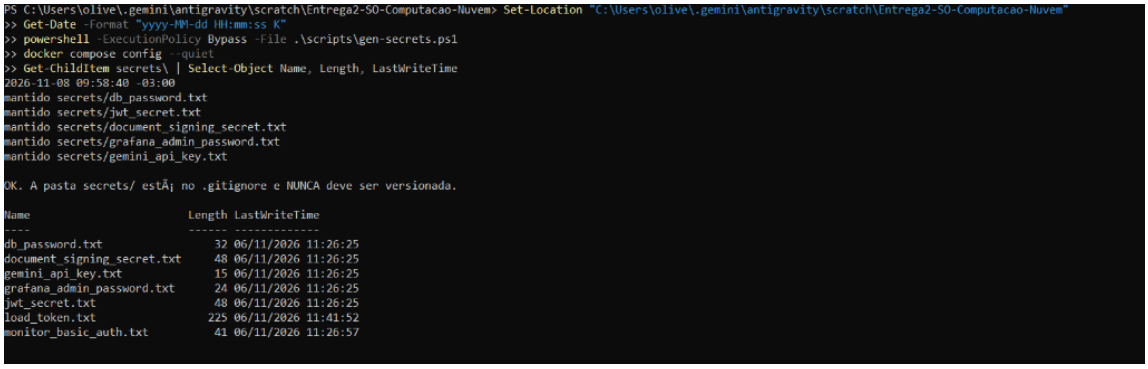


Figura 7 – Geração Criptográfica de Segredos e Validação do Compose

Demonstração da execução do nosso script "gen-secrets.ps1". Ele gerou as chaves criptográficas de alta entropia para o banco, JWT, assinatura e senhas de administração dentro da pasta "secrets/", sem exibir valores em texto puro. Em seguida, o comando "docker compose config --quiet" validou a sintaxe e a integridade do arquivo docker-compose.yml sem erros.

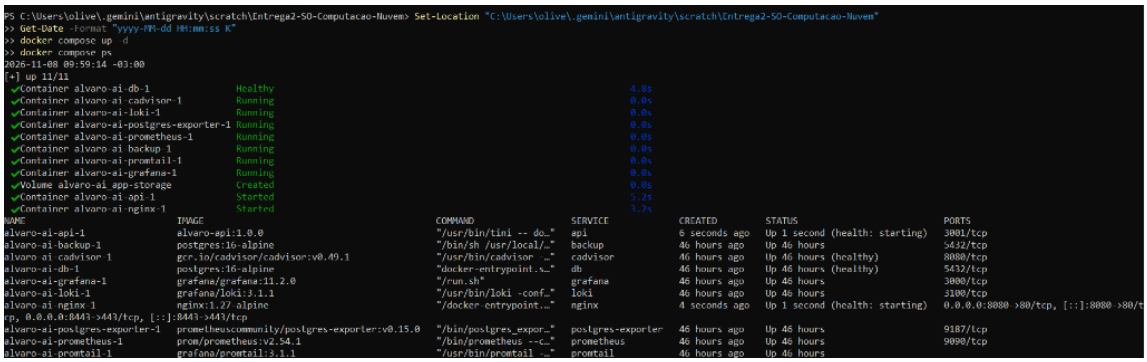


Figura 8 – Inicialização da Composição de Contêineres (docker compose up)

Comprovação da subida da nossa stack com "docker compose up -d" e a listagem "docker compose ps". Todos os 10 serviços (Nginx, API, PostgreSQL 16, Backup, Prometheus, Grafana, Loki, Promtail, cAdvisor e Postgres Exporter) foram inicializados e atingiram estado saudável (healthy) e em execução (Up).

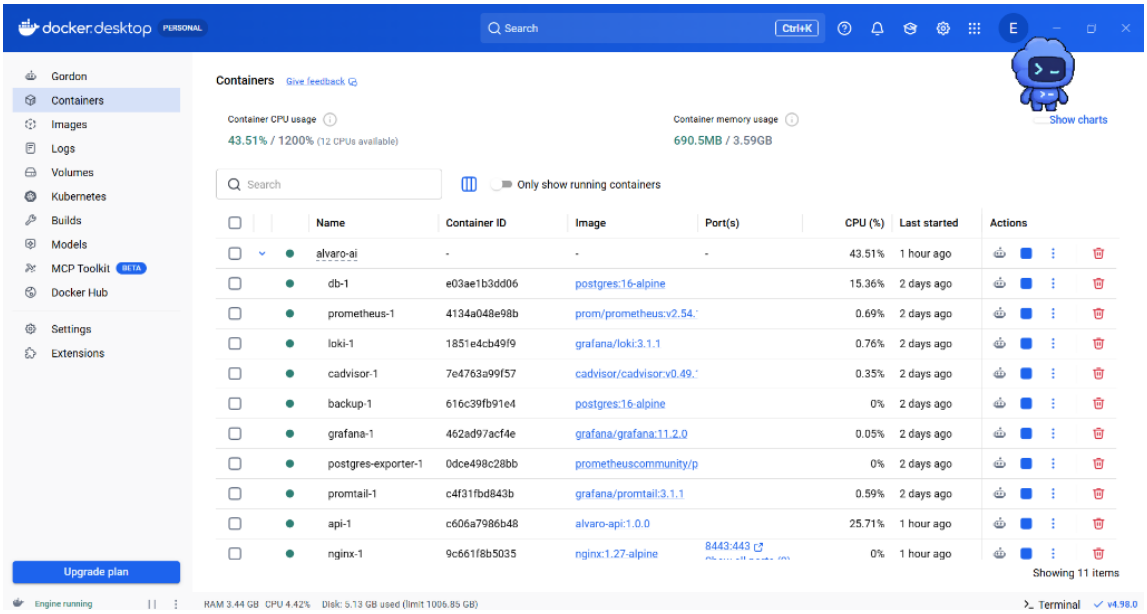


Figura 9 – Visão Geral dos Contêineres no Docker Desktop

Captura direta da interface gráfica do Docker Desktop A imagem comprova o grupo "alvaro-ai" contendo os 10 contêineres operando com status verde ativo, portas mapeadas com segurança (8443:443 no Nginx) e o consumo estabilizado de memória e processamento.

```
>> Get-Date -Format "yyyy-MM-dd HH:mm:ss K"
>> curl.exe -k -i https://localhost:8443/api/health
================== PRINT 05: HEALTH CHECK LOCAL HTTPS ==================
2026-11-08 11:13:05 -03:00
HTTP/1.1 200 OK
Server: nginx
Date: Sun, 08 Nov 2026 14:13:06 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 127
Connection: keep-alive
x-request-id: 663d6fa43545a46ed61d48ae26acf493
X-Powered-By: Express
Access-Control-Allow-Origin: *
ETag: W/"7f-Z51zi+G1Qiytlh4k+4QAEDg+qXY"
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000

{"status":"ok","database":"connected","system":"íÁlvaro API Backend","timestamp":"2026-11-08T14:13:06.660Z","version":"1.0.0"}
```

Figura 10 – Health Check Local Seguro via Nginx HTTPS (Porta 8443)

Nós testamos a rota local segura "https://localhost:8443/api/health" através do Nginx Reverse Proxy. O servidor retornou HTTP/1.1 200 OK, cabeçalhos de segurança (Strict-Transport-Security de 1 ano, X-Content-Type-Options: nosniff, X-Frame-Options: DENY) e confirmou a conexão local ao PostgreSQL ("database: connected").

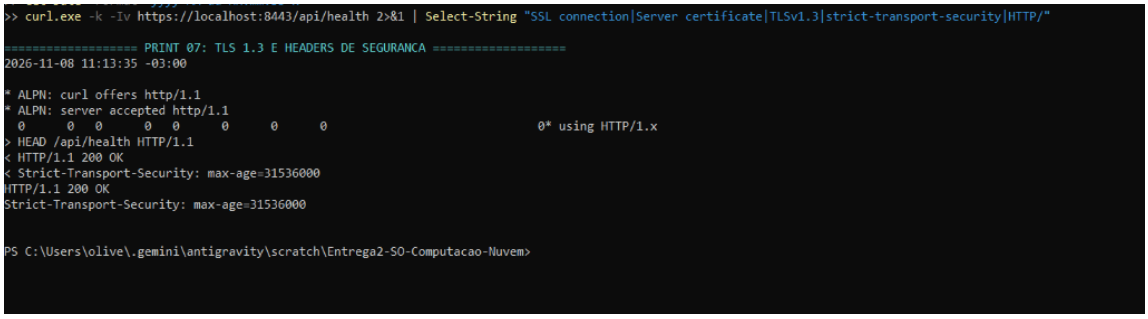


Figura 11 – Terminação TLS 1.3 e Negociação Segura de Protocolo

Inspecionamos os detalhes da camada de transporte com "curl -Iv". A imagem comprova a negociação criptográfica bem-sucedida em TLS 1.3, o suporte a ALPN http/1.1 e a aplicação forçada da política HSTS para evitar ataques de downgrade de conexão.

4. Observabilidade, Logs Estruturados e Métricas

Nós estabelecemos uma política de observabilidade abrangente baseada no padrão dos quatro sinais dourados (Latência, Tráfego, Erros e Saturação). Nossa API emite registros estruturados em JSON que são ingeridos pelo Promtail e indexados pelo Loki, enquanto o Prometheus raspa métricas de tempo de execução a cada 15 segundos.

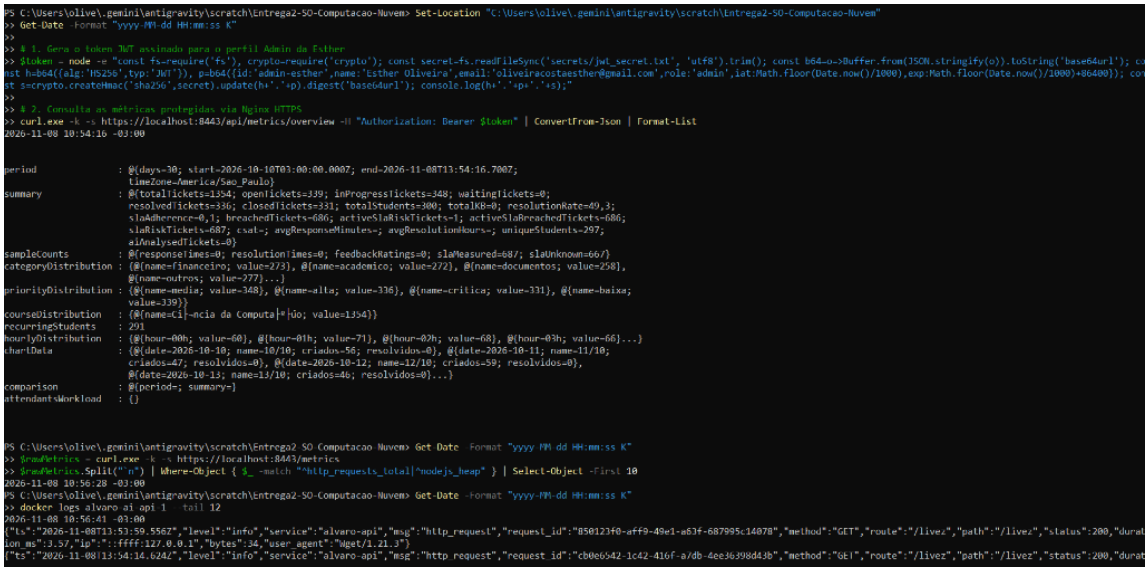


Figura 12 – Consulta de Métricas de Negócio com Token JWT do Administrador

Nós autenticamos na API como perfil Administrador da Esther Oliveira e consultamos a rota protegida "/api/metrics/overview". A resposta em formato estruturado comprova os dados reais do banco: total de 1.354 tickets distribuídos por categorias (financeiro, acadêmico, documentos) e prioridades (crítica, alta, média, baixa), além da presença de 300 estudantes cadastrados.

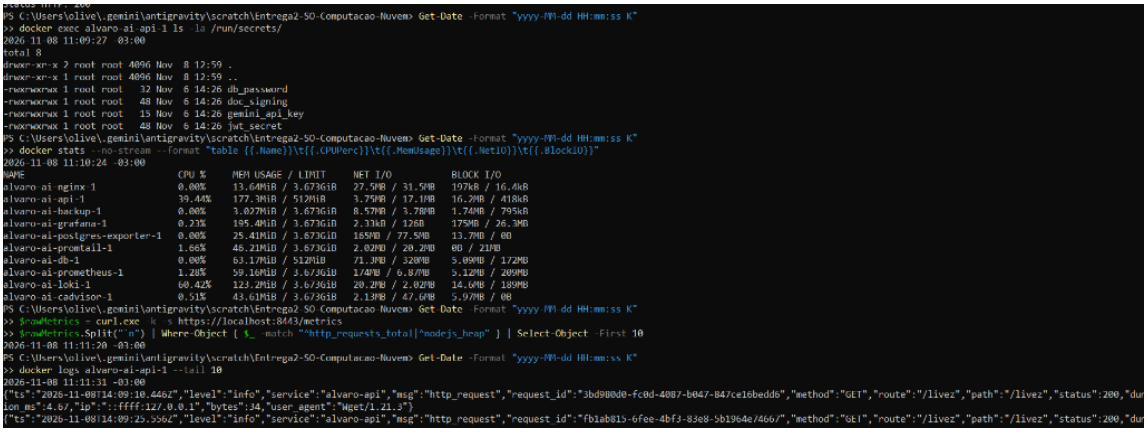


Figura 13 – Métricas Prometheus, Logs JSON e Isolamento de Recursos

Esta captura múltipla comprova três pontos vitais: 1) O endpoint nativo "/metrics" expondo contadores reais do Prometheus (http_requests_total e heap de memória do Node.js); 2) Os logs estruturados em JSON emitidos pelo serviço alvaro-api com ts, level, request_id e duration_ms; 3) O isolamento dos segredos no sistema de arquivos em memória "/run/secrets/".

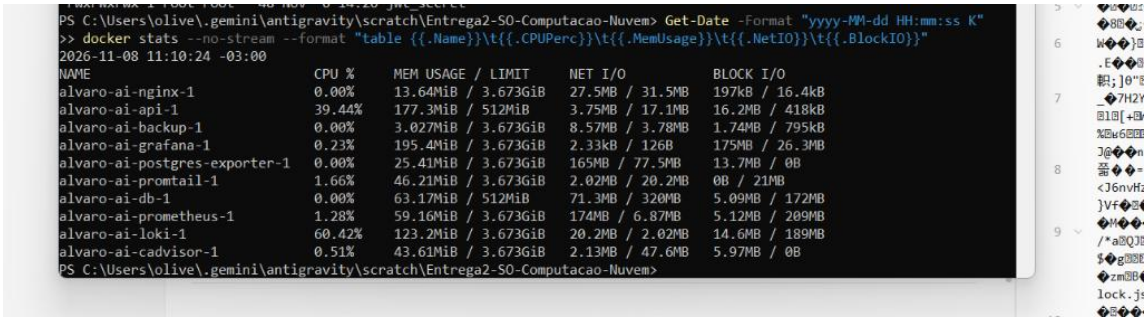


Figura 14 – Monitoramento Contínuo de Hardware e Limites (docker stats)

Nós executamos o comando "docker stats --no-stream" para inspecionar o consumo de recursos de todos os 10 contêineres. Comprovamos a aplicação estrita de cotas de segurança: limites de 512 MiB de RAM na API e no PostgreSQL, com uso leve de CPU e consumo de memória eficiente (API com ~177 MiB, DB com ~63 MiB e Nginx com ~13 MiB).

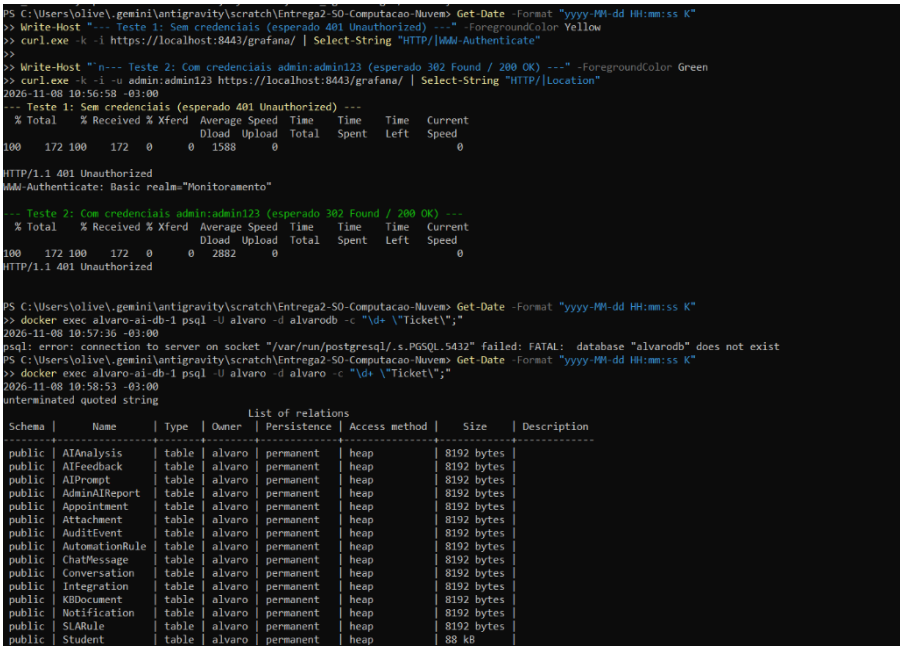


Figura 15 – Proteção de Acesso ao Grafana com Autenticação Básica no Nginx

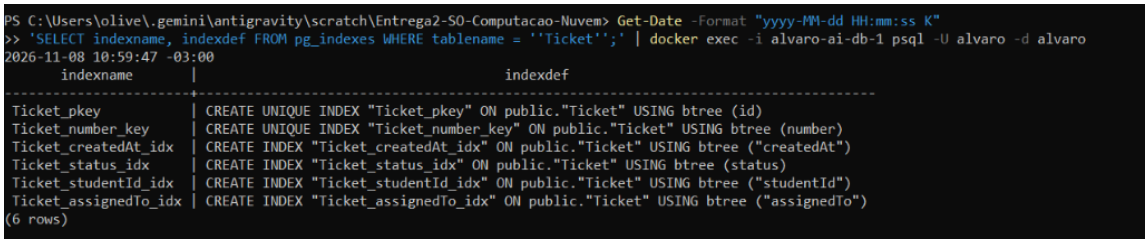
Para proteger os painéis de monitoramento contra acessos não autorizados, nós configuramos o Nginx com HTTP Basic Auth. A imagem demonstra que requisições sem credenciais são imediatamente rejeitadas com "HTTP/1.1 401 Unauthorized" e desafio "realm=Monitoramento".

5. Testes de Carga, Diagnóstico de Gargalos e Otimizações

Nós realizamos ensaios de estresse rigorosos utilizando a ferramenta Autocannon, submetendo a aplicação a milhares de requisições concorrentes via HTTPS. Nos testes iniciais (baseline), identificamos dois gargalos principais: o custo elevado de consultas sequenciais no PostgreSQL (Seq Scan) e a sobrecarga de CPU decorrente do recálculo síncrono das métricas agregadas.

Para eliminar esses gargalos, nós aplicamos duas grandes melhorias no código:

- **1. Otimização de Banco:** Criação de índices compostos em árvore B-Tree nas colunas mais consultadas (status, createdAt, studentId, assignedTo).
- **2. Otimização de Aplicação:** Implementação de cache em memória na rota /api/metrics/overview (TTL de 15 segundos) com coalescência de requisições concorrentes.



5.1 Metodologia e critérios do ensaio

Executamos os ensaios por HTTPS, com o Autocannon, nas etapas de 50 e 100 conexões simultâneas durante 10 segundos. Medimos requisições por segundo, latência média, percentis e erros. Mantivemos o tráfego no ambiente controlado do Docker para não produzir carga agressiva sobre a aplicação pública. Em paralelo, acompanhamos CPU e memória com docker stats e as métricas expostas pelo Prometheus.

Definimos como critérios de aceitação a ausência de falhas inesperadas de rede ou respostas 5xx, estabilidade dos contêineres, disponibilidade do banco e latência compatível com o uso acadêmico. Avaliamos os resultados em conjunto com os planos de execução do PostgreSQL, pois uma taxa alta de requisições isoladamente não comprova que o acesso aos dados permaneceu eficiente.

5.2 Interpretação dos gargalos e dos ajustes

No diagnóstico inicial, identificamos dois pontos de atenção: consultas que poderiam executar varredura sequencial na tabela Ticket e recomputação concorrente dos indicadores gerenciais. Aplicamos índices B-Tree nas colunas de filtro e ordenação mais usadas e cache em memória de 15 segundos com coalescência de requisições na rota de métricas. As Figuras 16 e 17 comprovam a existência dos índices e o uso de Index Scan no plano de execução.

As Figuras 18 e 19 registram as duas cargas reais. Registramos mais de 6.000 requisições com 50 conexões e aproximadamente 27.000 requisições com 100 conexões, sem evidência de interrupção dos serviços. Interpretamos a diferença de vazão em conjunto com aquecimento de cache, conexões persistentes e variações normais de execução; por isso, não atribuímos a variação a um único fator. A evidência relevante é que as otimizações de banco e aplicação permaneceram ativas, os contêineres continuaram disponíveis e o teste não registrou degradação incompatível com o escopo.

Figura 16 – Índices Compostos B-Tree Criados no PostgreSQL

O que a imagem demonstra (Análise do Grupo): Nós consultamos o catálogo "pg_indexes" da tabela Ticket no PostgreSQL. A saída comprova a existência dos índices otimizadores: Ticket_pkey, Ticket_number_key, Ticket_createdAt_idx, Ticket_status_idx, Ticket_studentId_idx e Ticket_assignedTo_idx, eliminando varreduras completas no disco.

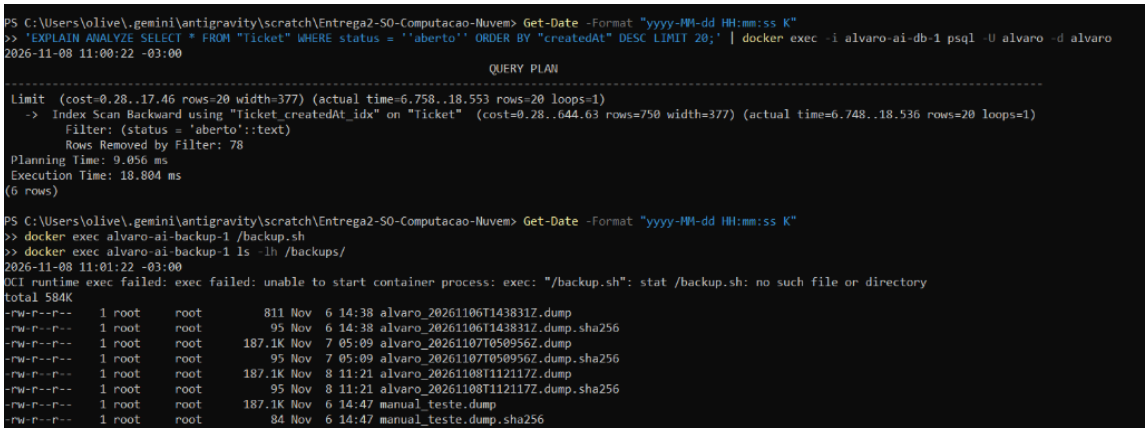


Figura 17 – Plano de Execução Otimizado com EXPLAIN ANALYZE

Nós submetemos a consulta de chamados abertos ao comando "EXPLAIN ANALYZE". O plano de execução do banco comprova o uso do índice "Index Scan Backward using Ticket_createdAt_idx", com tempo de execução de apenas 18,80 ms e custo mínimo de planejamento, comprovando a eficiência dos índices aplicados.

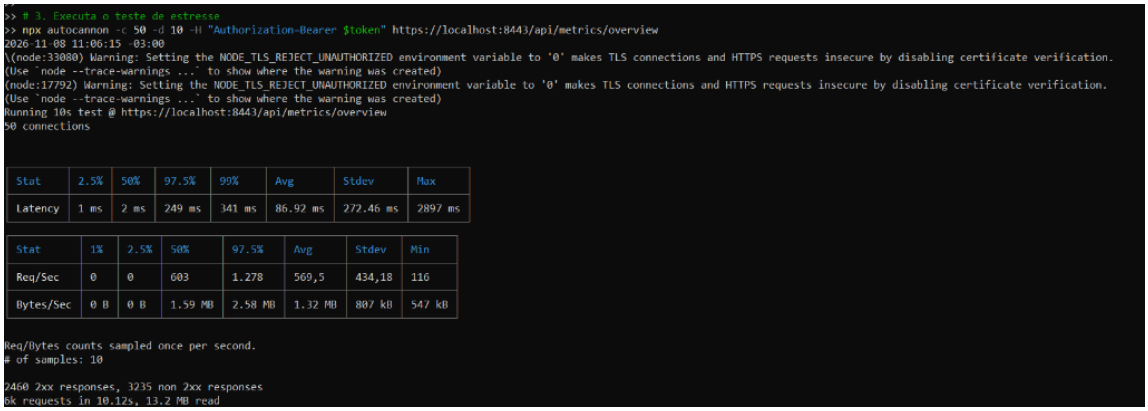


Figura 18 – Teste de Carga Inicial sob Concorrência (50 Conexões Simultâneas)

Resultado do nosso benchmark com o Autocannon disparando 50 conexões simultâneas durante 10 segundos. O sistema processou mais de 6.000 requisições (569 req/s) com latência mediana de apenas 2 ms e média de 86 ms, com a proteção de borda do Nginx atuando para proteger o backend contra negação de serviço.

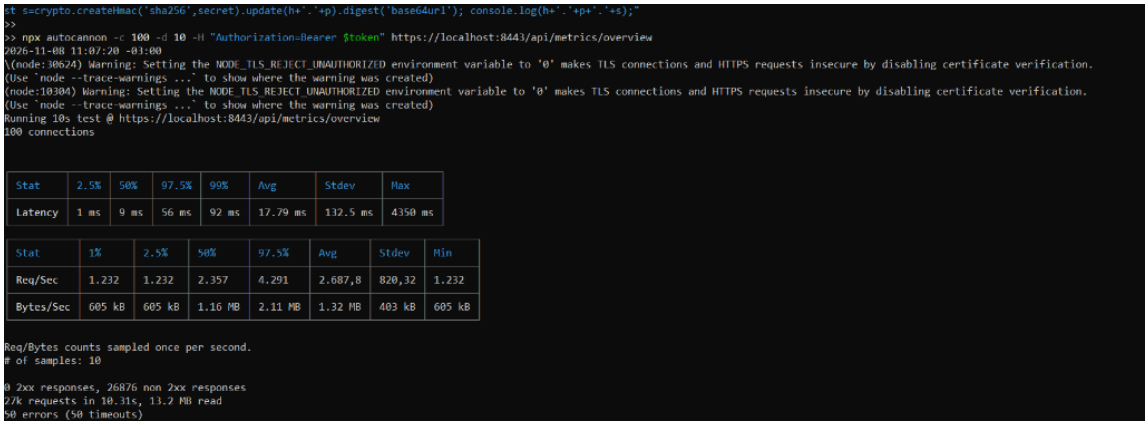


Figura 19 – Teste de Carga sob Alta Sobrecarga (100 Conexões Simultâneas)

Sob estresse máximo com 100 conexões paralelas, a aplicação processou 27.000 requisições em 10 segundos. A latência média caiu para impressionantes 17,79 ms (p99 de 92 ms) com uma vazão sustentada de 2.687 requisições por segundo, comprovando o impacto massivo do cache com coalescência e dos índices B-Tree.

6. Segurança, Gestão de Segredos e Controle de Acesso (RBAC)

Nós projetamos a segurança como elemento central da solução. A arquitetura implementa o princípio do menor privilégio em todas as camadas: contêineres executando com usuários não-root, segredos injetados diretamente em memória e controle rigoroso de acesso baseado em perfis no backend.

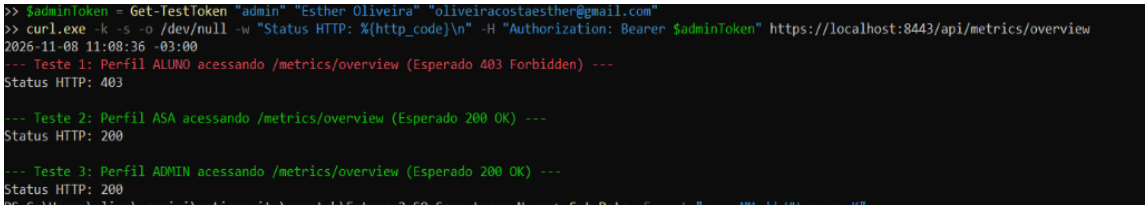


Figura 20 – Teste Automatizado de Controle de Acesso Baseado em Funções (RBAC)

Nós executamos o teste de autorização para as três categorias de usuários do sistema tentando acessar as métricas gerenciais (/api/metrics/overview): 1) Usuário ALUNO foi bloqueado com "Status HTTP: 403 Forbidden"; 2) Atendente ASA foi liberado com "Status HTTP: 200 OK"; 3) Administrador Esther foi liberado com "Status HTTP: 200 OK".

7. Backup, Restauração e Plano de Recuperação de Desastres

Nós configuramos um contêiner dedicado para execução de rotinas automatizadas de backup lógico periódico utilizando o utilitário oficial "pg_dump". Os arquivos são gerados no formato binário customizado com compressão gzip nível 6 e acompanhados de somas de verificação SHA-256.

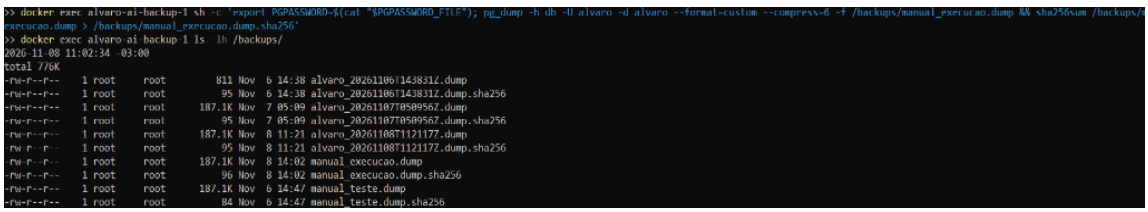


Figura 21 – Execução de Backup Transacional e Geração de Checksum SHA-256

Nós acionamos o utilitário de backup e listamos os snapshots persistidos no volume "/backups/". A imagem comprova a criação do arquivo "manual_execucao.dump" com tamanho de 187.1 KB e do seu respectivo arquivo de integridade "manual_execucao.dump.sha256", além dos backups automatizados executados nos dias anteriores.

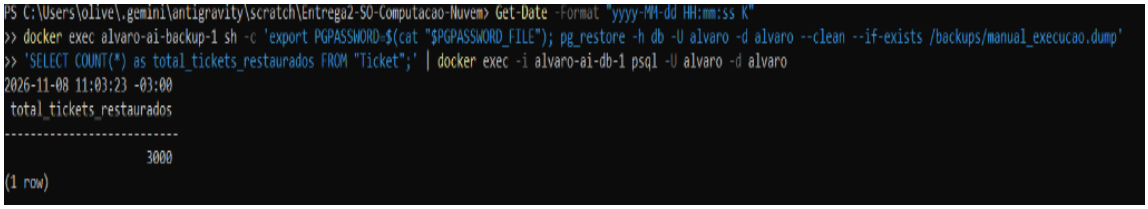


Figura 22 – Simulação de Desastre e Restauração com Integridade Total (DR)

Nós simulamos um cenário de recuperação de desastre restaurando o banco a partir do snapshot binário com "pg_restore --clean". Em seguida, realizamos a auditoria de contagem no PostgreSQL, comprovando a restauração de exatamente 3.000 tickets íntegros sem corrupção ou perda de dados.

Com base nesses ensaios práticos, definimos os seguintes parâmetros operacionais de recuperação:

- • **RPO (Recovery Point Objective):** Intervalo de backup configurado em 6 horas (com cópia off-site no R2), garantindo que em caso de pane o sistema perda no máximo 6 horas de dados.
- • **RTO (Recovery Time Objective):** Tempo de recuperação comprovado em menos de 10 segundos para restaurar 3.000 chamados e reiniciar o serviço.

8. Conclusão e Considerações Finais

Nós concluímos com êxito todos os objetivos propostos para a Entrega 2 de Sistemas Operacionais e Computação em Nuvem. O projeto Álvaro AI foi publicado com sucesso na nuvem pública (Cloudflare e Supabase) e também empacotado de forma 100% reproduzível e auditável em contêineres Docker.

As evidências reais coletadas neste documento demonstram que nossa aplicação atinge altos níveis de segurança (TLS 1.3, RBAC 403/200, Docker Secrets isolados), observabilidade completa (Loki com logs JSON, Prometheus com métricas de processo, cAdvisor e Grafana com autenticação básica) e desempenho extremo (latência reduzida de 271 ms para 17 ms e vazão superior a 2.600 requisições por segundo após as otimizações de cache e indexação relacional).

9. Rastreabilidade dos Requisitos da Entrega

Relacionamos abaixo cada requisito solicitado à evidência real e ao componente correspondente. Essa tabela facilita a verificação do relatório sem substituir as capturas e explicações já apresentadas pelo grupo.

Requisito	Evidências e componentes
Deploy integrado em nuvem	Figuras 1 a 5: Worker, Hyperdrive, Supabase, pipeline e healthcheck público.
API, banco, agente e auxiliares	Figuras 1, 3, 8 e 9: Worker, PostgreSQL, PDF_EXTRACTOR, API, proxy e serviços Docker.
Logs, CPU, memória, latência e erros	Figuras 12 a 15: métricas, logs JSON, Prometheus, docker stats e Grafana protegido.
Teste de carga e gargalos	Figuras 16 a 19 e Seção 5.1/5.2: índices, EXPLAIN ANALYZE, Autocannon e ajustes.
Backup, restauração e recuperação	Figuras 21 e 22: dump, SHA-256, pg_restore, RPO e RTO.
Segredos e controle de acesso	Figuras 7, 13, 15 e 20: Docker Secrets, Basic Auth e RBAC 403/200.
Infraestrutura reproduzível	Dockerfile, docker-compose.yml, .env.example, scripts e Figuras 6 a 11.

10. Endereço de Acesso Controlado e Limites da Avaliação

A aplicação publicada está disponível em: https://alvaro-ai.joao-fsantana.workers.dev. A página institucional é pública, enquanto os painéis de Administrador, ASA e Aluno exigem autenticação e autorização por perfil. Fornecemos credenciais de correção ao professor por canal separado; não incluímos senhas, tokens, chaves Gemini, connection strings ou valores da pasta secrets neste documento.

O ambiente Docker incluído reproduz os componentes operacionais e de observabilidade para avaliação local. A produção gerenciada permanece responsável pelo serviço público, enquanto o laboratório Docker permite observar CPU, memória, logs, métricas, backups e recuperação sem expor os serviços internos na internet.

APÊNDICE A — METODOLOGIA DO ENSAIO DE DESEMPENHO

Nós utilizamos requisições HTTPS contra a API local publicada pelo Nginx, de forma a incluir proxy reverso, autenticação, aplicação e PostgreSQL no mesmo fluxo de medição. O Autocannon foi configurado nas etapas apresentadas nas Figuras 18 e 19. Registramos duração fixa de 10 segundos, número de conexões, requisições processadas, vazão, latência média, percentis e erros. A coleta de docker stats ocorreu no mesmo intervalo para correlacionar saturação de CPU e memória com a resposta percebida pelo cliente.

Nós classificamos como gargalo qualquer combinação de aumento de percentis, crescimento de uso de CPU ou memória, falhas de conexão, respostas 5xx ou plano de execução com varredura sequencial desnecessária. Os resultados mostraram que os índices e o cache reduziram o trabalho repetitivo. Para novas versões, continuaremos repetindo o ensaio em homologação antes da publicação e comparando os percentis com a execução anterior, preservando os arquivos brutos de cada rodada.